

E4UnityIntegration-MIT: An Open-Source Unity Plug-in for Collecting Physiological Data Using Empatica E4 during Gameplay

Eduard de Vidal
Massachusetts Institute of Technology
Cambridge, Massachusetts, USA
eduarddv@mit.edu

Caglar Yildirim
Massachusetts Institute of Technology
Cambridge, Massachusetts, USA
caglary@mit.edu

Fox D. Harrell
Massachusetts Institute of Technology
Cambridge, Massachusetts, USA
fox.harrell@mit.edu

ABSTRACT

Physiological measurement of player experience (PX) during gameplay has been of increasing interest within game research circles. A commonly-used non-invasive wearable device for physiological measurement is the Empatica E4 wristband, which offers multiple physiological metrics, ranging from electrodermal activity to heart rate. That said, the E4's integration with popular game engines such as Unity 3D presents certain challenges due to non-obvious critical bugs in the library and limited documentation applicability within the Unity context. In this paper, we present an open-source Unity plug-in designed to mitigate the challenges associated with integrating the E4 into Unity projects: E4UnityIntegration-MIT. The plug-in exposes the E4's API for interfacing with Unity C# scripts, thereby enabling realtime data collection and monitoring. E4UnityIntegration-MIT also provides the affordance of saving the E4 data into an external file for data analysis purposes.

CCS CONCEPTS

• Human-centered computing → User studies.

KEYWORDS

Empatica E4, psychophysiological measures, affective gaming, Unity, plugin

ACM Reference Format:

Eduard de Vidal, Caglar Yildirim, and Fox D. Harrell. 2023. E4UnityIntegration-MIT: An Open-Source Unity Plug-in for Collecting Physiological Data Using Empatica E4 during Gameplay. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3586182.3616627>

1 INTRODUCTION

Assessing player experience (PX) is a matter of importance in game research. There are a range of approaches to measuring PX, from more subjective methods such as interviews or questionnaires to more objective techniques like in-game analytics and psychophysiological measurements. The latter offers advantages such as not needing to interrupt the experience, nor introducing memory bias

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3616627>

(by not requiring of players to recall their experience after the fact) [3].

Common physiological measurements taken include electrodermal activity (EDA), heart rate (HR), interbeat interval (IBI), heart rate variability (HRV), blood volume pulse (BVP), temperature, electroencephalography (EEG), electromyography (EMG), respiratory rate (RR) and eye tracking [3]. These metrics are obtained through the use of a range of commercial or research devices. One such commonly used device [1, 4, 5], is the Empatica E4 (<https://www.empatica.com/research/e4/>), which is a non-invasive wearable device for capturing physiological data. The E4 captures BVP, EDA, IBI, and HR and provides access to the data through APIs for Android and iOS platforms.

The availability of an Android API, the E4 link library, is particularly interesting due to the plethora of devices which run on an Android or Android-based operating system, many of which are suitable platforms for games. Indeed, even some virtual reality (VR) headsets such as Meta's Quest offerings are Android-based. Thus, there is an increasing interest in and need for integrating the E4 wristband into game engines that support Android-based game releases. One such game engine that is commonly used and supported by the E4 API is the Unity 3D game engine (referred to as Unity). However, it is not straightforward to use the E4 link library in Unity.

This paper presents an open-source Unity plug-in for capturing data from the E4 wristband in Unity: E4UnityIntegration-MIT. The plug-in was designed as a minimal plug-in that exposes the API for interfacing in Unity C# scripts to enable game researchers to quickly integrate the E4 wristband into their Unity projects and to capture the various types of physiological data it provides. E4UnityIntegration-MIT allows for receiving the E4's data streams in real-time, thus enabling the creation of physiological feedback loops, while at the same providing the functionality to save the received data into a file.

2 PLUG-IN DESIGN

The desired functionality was to support the E4 within Unity, exposing the API to Unity C# scripts, with special attention being paid to allowing the data streams to be received and acted upon in real-time, as well as be saved to a file.

Additionally, the plug-in was designed to have a simple installation and minimal dependencies. The justification is that when choosing which sensors to use for a playtesting session or user study, it is often important to have proof of the feasibility of using a candidate device. In this case, researchers considering using the

E4 may wish to quickly check if the E4 can be satisfactorily integrated into their Unity project. Thus, E4UnityIntegration-MIT was made so that it can be installed by merely dropping the files into the project's plug-ins folder. Moreover, the plug-in has no dependencies besides Empatica's E4 link.

3 TECHNICAL DESCRIPTION

3.1 Installation instructions

It is simple to install E4UnityIntegration-MIT. All its files must be placed in the Assets/Plugins/Android directory of a given Unity project. Then the following steps should be followed:

- Enable Custom Main Manifest in the project settings. If it is already enabled and a custom main manifest is already in use, then the plug-in must be merged into the existing one, by copying over the lines related to permissions and feature requirements.
- Enable Custom Main Gradle Template in the project settings. If it is already enabled and a custom main gradle template is already in use, then the plug-in must be merged into the existing one, by copying over the *okhttp* dependency (required by Empatica's E4 link).

Lastly, needless to state, Empatica's E4 link is also needed. It is available for download through Empatica's E4 connect web portal for developers (<https://e4.empatica.com/connect/developer.php>).

3.2 Plug-in overview

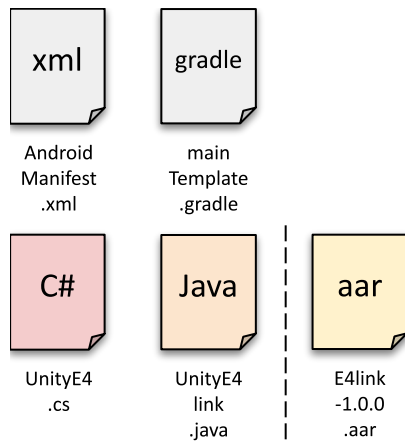


Figure 1: E4UnityIntegration-MIT files and dependencies. At the top are the build and configuration files that must be used or merged with existing ones. At the bottom, left of the dashed line, are the source files that may be edited by users to achieve desired functionality. To the right of the dashed line is Empatica's E4 link library; it is not distributed with the plug-in and must be obtained directly from Empatica.

The plug-in is available for download online [6]. It is provided in the form of open-source code files; users can use the plug-in by editing them directly. The provided code's behavior is to connect to the first E4 it encounters, which we expect is a useful default. Following is an overview of each file's purpose and content.

3.2.1 UnityE4.cs. Unity C# script containing the UnityE4 C# class. Attach the script to an empty GameObject to use the plug-in.

Inside, the Empatica API is exposed by providing two inner classes, *UnityE4LinkDataDelegateCallback* and *UnityE4LinkStatusDelegateCallback*, which implement the E4 link interfaces for receiving data and status updates, respectively. Users wishing to integrate the plug-in to their use cases should alter the implementation of these classes' callback methods accordingly.

Please note that callback code may not execute Unity API calls. Unity limits API calls to running on its main thread. Instead, the callbacks are called by the E4 link library's code, which runs on a different thread. To react to received data or status updates, store the information and trigger the desired behavior on the following Unity Update. Additionally, there is also code to record the received data at an interval (default is every .25 seconds) and store it in a file.

3.2.2 UnityE4Link.java. Auxiliary Java file containing the *UnityE4Link* class. It serves as a necessary bridge between E4 link and *UnityE4.cs*.

Having to work around a bug in the Empatica library has also shaped the plug-in's design. E4 link exposes two Java interfaces, one to receive the E4's status updates and another to receive its data updates. Both are essential for the usage of the library. However, there exists a bug whereby a method from one interface is called on an object whose class implements the other interface¹. The bug is critical in that it causes the application to instantly crash if allowed to happen. The workaround is to have one same class implement both interfaces. However, the C# class that Unity provides to implement Java interfaces (*AndroidJavaProxy*) is only able to implement one at a time. As such, it is unfortunately not possible to have all of the plug-in's code in a single C# script. We have therefore opted to implement the workaround in this separate file.

Introducing an intermediary Java file is not without advantages, however. The present implementation allows programming E4 link related functionalities directly in Java, which may result in simpler code considering the alternative is having to call Java's utilities from C#. For example, the provided default behavior of connecting to the first E4 encountered is contained in this file.

3.2.3 AndroidManifest.xml. The plug-in's custom main manifest. Specifies the need for Bluetooth, internet, and location permissions, as well as the Bluetooth low energy feature.

3.2.4 mainTemplate.gradle. The plug-in's custom main gradle template. Specifies E4 link's *okhttp* dependency.

4 USE CASE AND APPLICATIONS

The intended user base for this plug-in are game researchers and developers who use Unity for game development and research. Its potential applications include physiological data gathering, as in [1], or the creation of physiological feedback loops, as in [4]. Both of these studies employed the E4, demonstrating the interest and suitability of the device for these uses.

¹This bug was reported (on September 2020) and acknowledged by Empatica (on November 2020) [2] in the E4 link sample Android project repository. The issue was closed as completed on November 2020. Nevertheless, the latest version of E4 link available for download is from February 2020, before the report, and the bug is still present.

5 CONCLUSION

E4UnityIntegration-MIT is an open-source Unity plug-in designed for game researchers and developers to integrate Empatica's E4 wristband into Unity projects. Its intended use cases include physiological data collection and storage as well as development of physiological feedback loops in games. We have developed the plug-in in response to the difficulty in integrating Empatica's Android E4 link library into Unity and have published it open-source to enable the game research community to easily integrate it into their projects.

REFERENCES

- [1] Aaron Frederick Bulagang, James Mountstephens, and Jason Teo. 2021. Multiclass emotion prediction using heart rate and virtual reality stimuli. *Journal of Big Data* 8 (2021), 1–12.
- [2] Arne Herdick. 2020. Native Code expects "didUpdateOnWristStatus" in "EmpaDataDelegate". <https://github.com/empatica/e4link-sample-project-android/issues/2>.
- [3] Lennart E Nacke. 2013. An introduction to physiological player metrics for evaluating games. *Game Analytics: Maximizing the value of player data* (2013), 585–619.
- [4] Helena Polman. 2022. The Impact of Changing Moods Based on Real-Time Biometric Measurements on Player Experience. In *Gaming, Simulation and Innovations: Challenges and Opportunities: 52nd International Simulation and Gaming Association Conference, ISAGA 2021, Indore, India, September 6–10, 2021, Revised Selected Papers*. Springer, 171–181.
- [5] Elton Sarmanho Siqueira, Marcos Cordeiro Fleury, Marcus Vinicius Lamar, Anders Drachen, Carla Denise Castanho, and Ricardo Pezzuol Jacobi. 2022. An automated approach to estimate player experience in game events from psychophysiological data. *Multimedia Tools and Applications* (2022), 1–32.
- [6] Caglar Yildirim. 2023. An Android Plugin for E4 Wristband Integration in Unity. https://osf.io/v9whk/?view_only=dc43354770044134a45c0a74c312514f.